Database Development 158.337

Week 11

The Little MongoDB Book by Karl Seguin

Chapter 14

HDFS

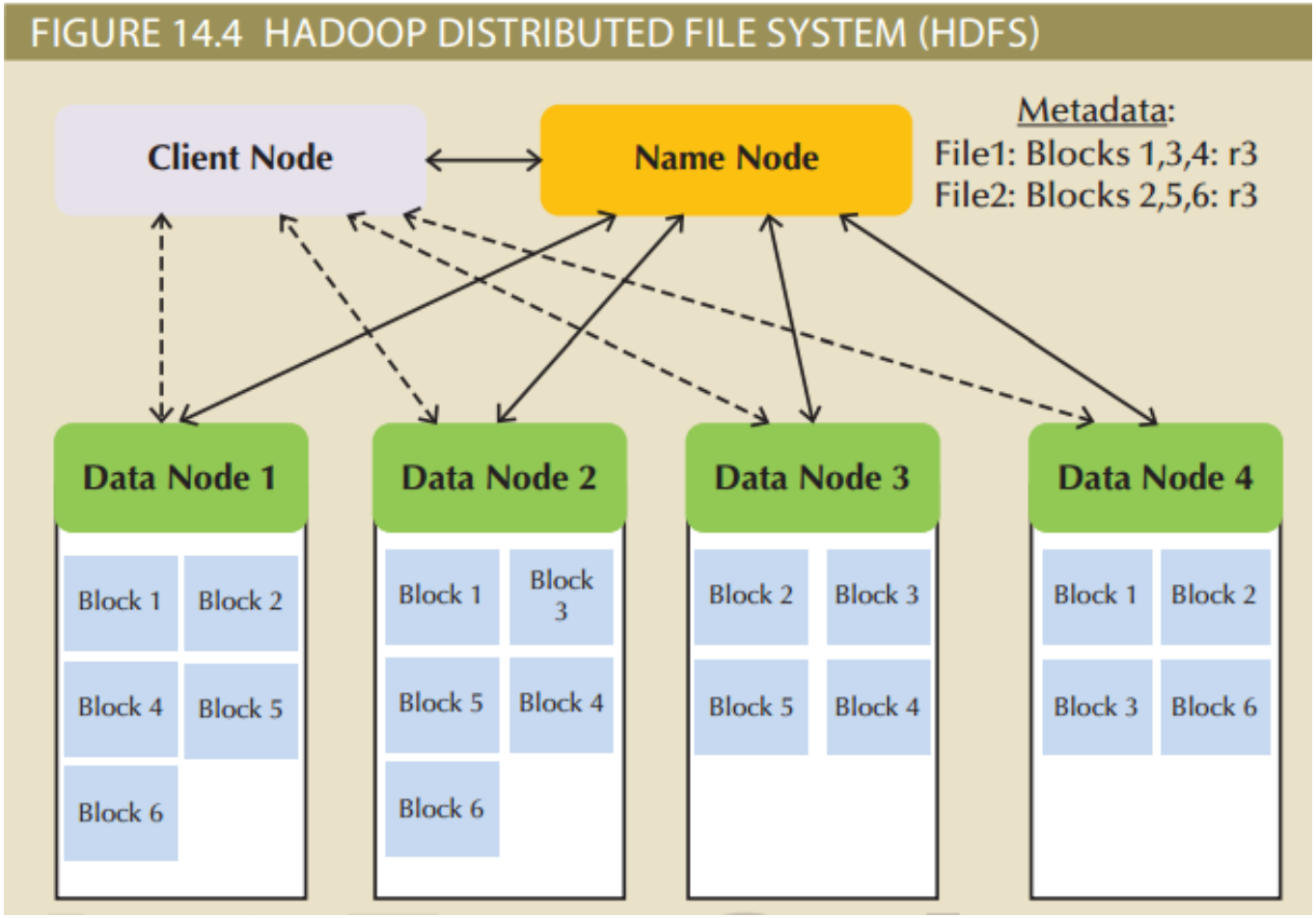
- Stores large files across multiple servers
  - MapReduce function
  - Parallel processing
  - Data mining analytics
  - ……

  (used by Yahoo search engine, Facebook, Amazon)

HDFS

- Hadoop uses several types of nodes: computers that perform one or more types of tasks within the system
  - Data node store the actual file data
  - Name node contains file system metadata
  - Client node makes requests to the file system as needed to support user applications

HDFS


FIGURE 14.4 HADOOP DISTRIBUTED FILE SYSTEM (HDFS)
Client Node
Name Node
Metadata:
File1: Blocks 1,3,4: r3
File2: Blocks 2,5,6: r3
Data Node 1
Block 1
Block 2
Block 4
Block 5
Block 6
Data Node 2
Block 1
Block 3
Block 5
Block 4
Block 6
Data Node 3
Block 2
Block 3
Block 5
Block 4
Data Node 4
Block 1
Block 2
Block 3
Block 6

MapReduce

- MapReduce is an open-source application programming interface (API) that provides fast data analytics services.

- MapReduce distributes the processing of the data among thousands of nodes in parallel.

- MapReduce works with structured and non-structured data.

MapReduce Processing

1. A client node submits a job to the Job Tracker.
2. Job Tracker determines where the data to be processed resides.
3. Job Tracker contacts the Task Tracker on the nodes as close as possible to the data.
4. Each Task Tracker creates mappers and reducers as needed to complete the processing of each block of data and consolidate that data into a result.
5. Task Trackers report results back to the Job Tracker when the mappers and reducers are finished.
6. The Job Tracker updates the status of the job to indicate when it is complete.

Customer

CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE	CUS_BALANCE
10010	Ramas	Alfred	A	615	844-2573	0
10011	Dunne	Leona	K	713	894-1238	0
10012	Smith	Kathy	W	615	894-2285	345.86
10013	Olowski	Paul	F	615	894-2180	536.75
10014	Orlando	Myron	X	615	222-1672	0
10015	O'Brian	Amy	B	713	442-3381	0
10016	Brown	James	G	615	297-1228	221.19
10017	Williams	George	X	615	290-2556	768.93
10018	Farriss	Anne	G	713	382-7185	216.55
10019	Smith	Olette	K	615	297-3809	0

Invoice

INV_NUMBER	CUS_CODE	INV_DATE
1001	10014	16/01/08
1002	10011	16/01/08
1003	10012	16/01/08
1004	10011	17/01/08
1005	10018	17/01/08
1006	10014	17/01/08
1007	10015	17/01/08
1008	10011	17/01/08
1009	10011	17/01/20

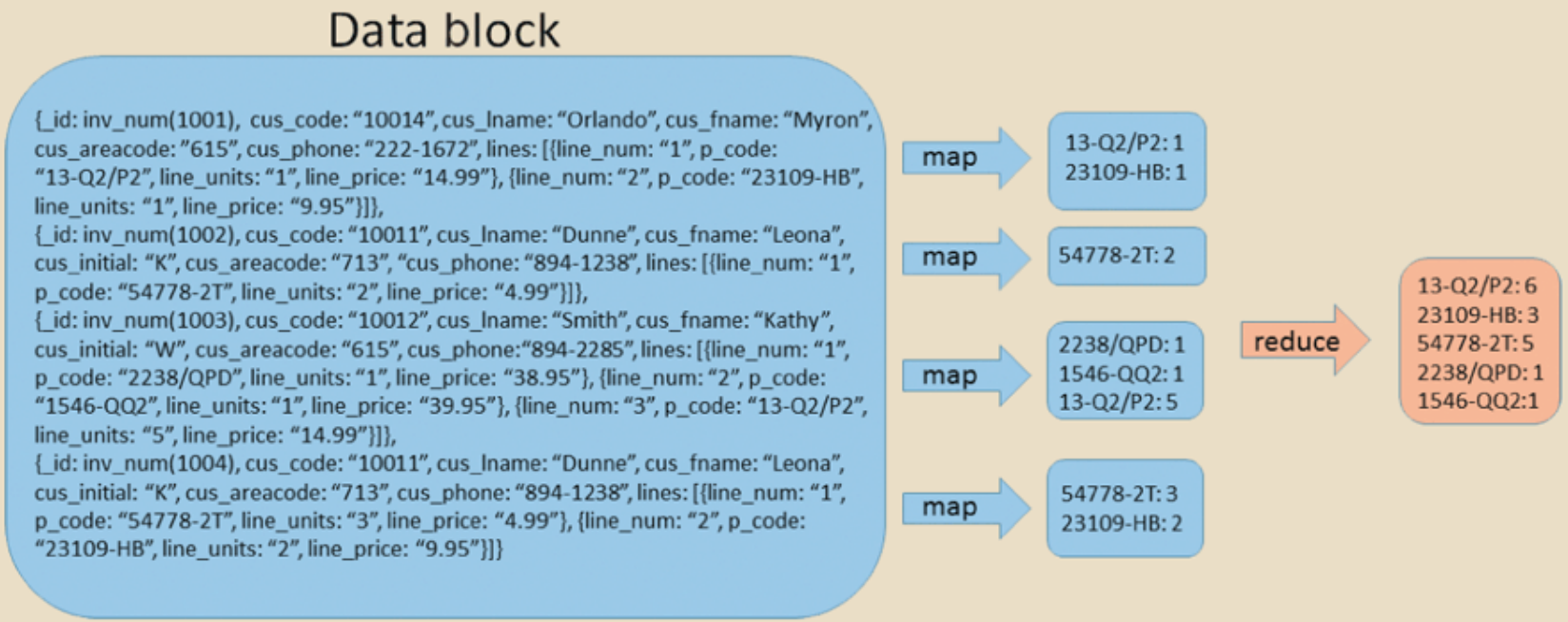
Line

INV_NUMBER	LINE_NUMBER	P_CODE	LINE_UNITS	LINE_PRICE
1001	1	13-Q2/P2	1	14.99
1001	2	23109-HB	1	9.95
1002	1	54778-2T	2	4.99
1003	1	2238/QPD	1	38.95
1003	2	1546-QQ2	1	39.95
1003	3	13-Q2/P2	5	14.99
1004	1	54778-2T	3	4.99
1004	2	23109-HB	2	9.95
1005	1	PVC23DRT	12	5.87
1006	1	SM-18277	3	6.99
1006	2	2232/QTY	1	109.92
1006	3	23109-HB	1	9.95
1006	4	89-WRE-Q	1	256.99
1007	1	13-Q2/P2	2	14.99
1007	2	54778-2T	1	4.99
1008	1	PVC23DRT	5	5.87
1008	2	WR3/TT3	3	119.95
1008	3	23109-HB	1	9.95
1009	1	13-Q2/P2	2	14.99

MapReduce

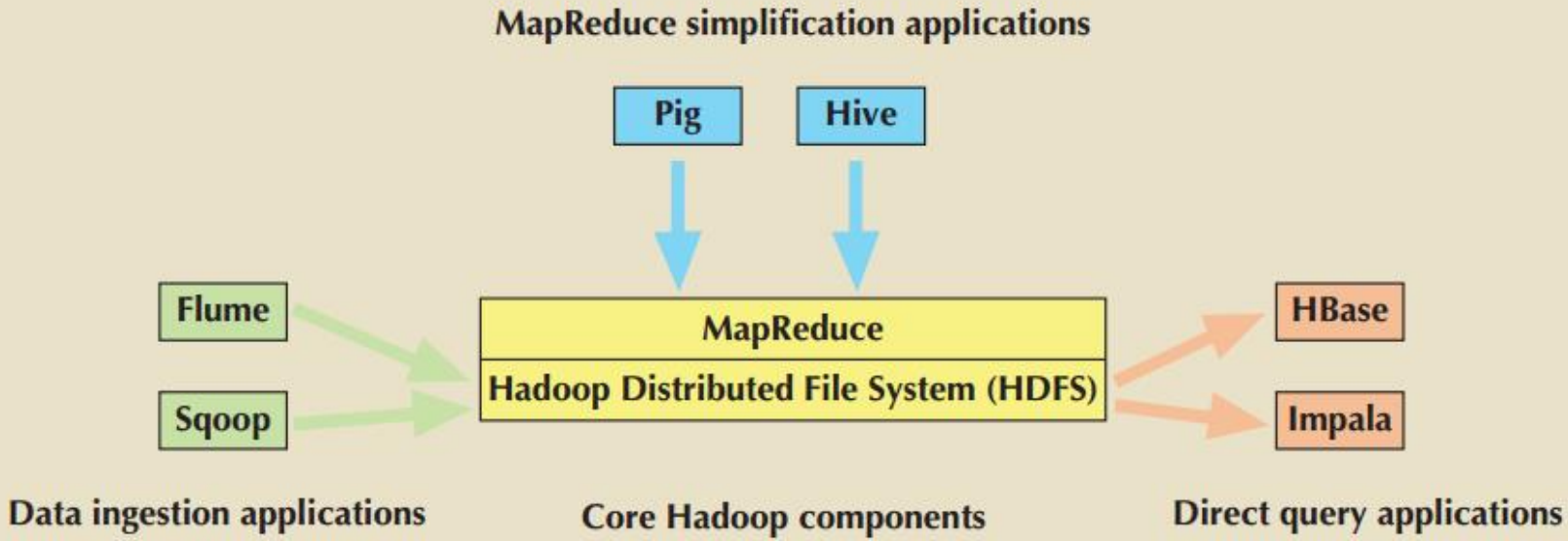
How many products have been sold?

FIGURE 14.5 MAPREDUCE



Hadoop (7 of 8)

FIGURE 14.6 A SAMPLE OF THE HADOOP ECOSYSTEM



MapReduce

The MapReduce framework provides two main functions, Map and Reduce.

- Map function takes a job and divides it into smaller units of work
- Reduce function collects all the output results generated from the nodes and integrates them into a single result set.

Map Reduce

Pattern

- Run mapReduce function on the collection

  2 step process

  – Map => emits an array of values based on a key

  – Reduce => takes above values and outputs the result

- Reduce is idempotent (applies regression principle – i.e., calling it multiple times will generate the same result as calling it once)

Map Reduce

Two-step process

1. Map - The mapping step transforms the inputted documents and emits a key=>value pair (the key and/or value can be complex). Then, key/value pairs are grouped by key, such that values for the same key end up in an array.

2. Reduce - The reduce gets a key and this array of values emitted for that key, and produces the final result. We'll look at each step, and the output of each step.

Example

- Here, we will rely on a hits collection with two fields: resource and date.

- Our desired output is a breakdown by resource, year, month, day and count.

- Count the hits on different web resources/web pages (e.g., index.html, about.html, contact_us.html)

Hello World for mapReduce

Hits on pages

(index and about)

resource	date
index	Jan 20 2010 4:30
index	Jan 20 2010 5:30
about	Jan 20 2010 6:00
index	Jan 20 2010 7:00
about	Jan 21 2010 8:00
about	Jan 21 2010 8:30
index	Jan 21 2010 8:30
about	Jan 21 2010 9:00
index	Jan 21 2010 9:30
index	Jan 22 2010 5:00



Reduced info

resource	year	month	day	count
index	2010	1	20	3
about	2010	1	20	1
about	2010	1	21	3
index	2010	1	21	2
index	2010	1	22	1

Key Value pairs

```
resource      date
index         Jan 20 2010 4:30
index         Jan 20 2010 5:30
```

ObjectID	Key	Value
12345	resource	index
12345	date	Date(2010, 1, 20, 4, 30)
12346	resource	index
12346	date	Date(2010, 1, 20, 5, 30)
12347	…	….

db.hits.insert({resource: "index", date: new Date(2010, 1, 20, 4, 30)});

db.hits.insert({resource: "index", date: new Date(2010, 1, 20, 5, 30)});

etc….

Map

- The goal of map is to make it emit a value which can be reduced. The map loops through each document.

- It’s possible for map to emit 0 or more times. In our case, it’ll always emit once (which is common).

- The value emitted depends on what we are calculating – count, average, sum, etc… (group functions)

Map

```
var map = function()
{
var key = {resource: this.resource,
year:
this.date.getFullYear(),month:

this.date.getMonth(),day:
this.date.getDate()};
emit(key, {count: 1});
};
```

Reduce

```
var reduce = function(key, values)
{
var sum = 0;
values.forEach(function(value) {
sum += value["count"];
});
return {count: sum};
};
```



mapReduce

```
db.hits.mapReduce(map, reduce, {out: "hits_stats"});
db.hits_stats.find();
```

Map

{resource: ’index’, year: 2010, month: 0, day: 20} => [{count: 1}, {count: 1}, {count:1}]

{resource: ’about’, year: 2010, month: 0, day: 20} => [{count: 1}]

{resource: ’about’, year: 2010, month: 0, day: 21} => [{count: 1}, {count: 1}, {count:1}]

{resource: ’index’, year: 2010, month: 0, day: 21} => [{count: 1}, {count: 1}]

{resource: ’index’, year: 2010, month: 0, day: 22} => [{count: 1}]

Reduce

{resource: ’index’, year: 2010, month: 0, day: 20} => {count: 3}

{resource: ’about’, year: 2010, month: 0, day: 20} => {count: 1}

{resource: ’about’, year: 2010, month: 0, day: 21} => {count: 3}

{resource: ’index’, year: 2010, month: 0, day: 21} => {count: 2}

{resource: ’index’, year: 2010, month: 0, day: 22} => {count: 1}

MapReduce

- We can pass our map and reduce functions to the mapReduce command by running:

```
db.hits.mapReduce(map, reduce, {out: 'hit_stats'});
db.hit_stats.find();
```

- Results will be stored in a collection called hit_stats

```
db.hit_stats.find();
```

Analytics

- The nice thing about this type of approach to analytics is that by storing the output, reports are fast to generate and data growth is controlled (per resource that we track, we'll add at most 1 document per day).

resource	year	month	day	count
index	2010	1	20	3
about	2010	1	20	1
about	2010	1	21	3
index	2010	1	21	2
index	2010	1	22	1

Aggregate Aware

- The designer of the database has to be aware of the way the data in the database will be used.
- The designer can then design the database around whichever component would be central to that usage.
- So, instead of decomposing the data structures to eliminate redundancy, an aggregate aware database collects, or aggregates, the data around a central component to minimize the structures required during processing.

Aggregation

https://www.mongodb.com/docs/manual/reference/map-reduce-to-aggregation-pipeline/

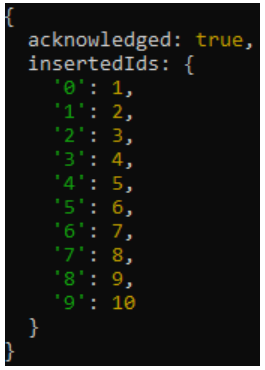
Map-reduce operations can be rewritten using aggregation pipeline operators, such as $group, $sum

I will use examples given here –

https://www.mongodb.com/docs/manual/tutorial/map-reduce-examples/

Example

```
db.orders.insertMany([
  { _id: 1, cust_id: "Ant O. Knee", ord_date: new Date("2020-03-01"), price: 25, items: [ { sku: "oranges", qty: 5, price: 2.5 }, { sku: "apples", qty: 5, price: 2.5 } ], status: "A" },
  { _id: 2, cust_id: "Ant O. Knee", ord_date: new Date("2020-03-08"), price: 70, items: [ { sku: "oranges", qty: 8, price: 2.5 }, { sku: "chocolates", qty: 5, price: 10 } ], status: "A" },
  { _id: 3, cust_id: "Busby Bee", ord_date: new Date("2020-03-08"), price: 50, items: [ { sku: "oranges", qty: 10, price: 2.5 }, { sku: "pears", qty: 10, price: 2.5 } ], status: "A" },
  { _id: 4, cust_id: "Busby Bee", ord_date: new Date("2020-03-18"), price: 25, items: [ { sku: "oranges", qty: 10, price: 2.5 } ], status: "A" },
  { _id: 5, cust_id: "Busby Bee", ord_date: new Date("2020-03-19"), price: 50, items: [ { sku: "chocolates", qty: 5, price: 10 } ], status: "A"},
  { _id: 6, cust_id: "Cam Elot", ord_date: new Date("2020-03-19"), price: 35, items: [ { sku: "carrots", qty: 10, price: 1.0 }, { sku: "apples", qty: 10, price: 2.5 } ], status: "A" },
  { _id: 7, cust_id: "Cam Elot", ord_date: new Date("2020-03-20"), price: 25, items: [ { sku: "oranges", qty: 10, price: 2.5 } ], status: "A" },
  { _id: 8, cust_id: "Don Quis", ord_date: new Date("2020-03-20"), price: 75, items: [ { sku: "chocolates", qty: 5, price: 10 }, { sku: "apples", qty: 10, price: 2.5 } ], status: "A" },
  { _id: 9, cust_id: "Don Quis", ord_date: new Date("2020-03-20"), price: 55, items: [ { sku: "carrots", qty: 5, price: 1.0 }, { sku: "apples", qty: 10, price: 2.5 }, { sku: "oranges", qty: 10, price: 2.5 } ], status: "A" },
  { _id: 10, cust_id: "Don Quis", ord_date: new Date("2020-03-23"), price: 25, items: [ { sku: "oranges", qty: 10, price: 2.5 } ], status: "A" }
]);
```



```
test> db.orders.find();
[
  {
    _id: 1,
    cust_id: 'Ant O. Knee',
    ord_date: ISODate("2020-03-01T00:00:00.000Z"),
    price: 25,
    items: [
      { sku: 'oranges', qty: 5, price: 2.5 },
      { sku: 'apples', qty: 5, price: 2.5 }
    ],
    status: 'A'
  },
  {
    _id: 2,
    cust_id: 'Ant O. Knee',
    ord_date: ISODate("2020-03-08T00:00:00.000Z"),
    price: 70,
    items: [
      { sku: 'oranges', qty: 8, price: 2.5 },
      { sku: 'chocolates', qty: 5, price: 10 }
    ],
    status: 'A'
  },
  {
    _id: 3,
    cust_id: 'Busby Bee',
    ord_date: ISODate("2020-03-08T00:00:00.000Z"),
    price: 50,
    items: [
      { sku: 'oranges', qty: 10, price: 2.5 },
      { sku: 'pears', qty: 10, price: 2.5 }
    ],
    status: 'A'
  },
  {
    _id: 4,
    cust_id: 'Busby Bee',
    ord_date: ISODate("2020-03-18T00:00:00.000Z"),
    price: 25,
    items: [ { sku: 'oranges', qty: 10, price: 2.5 } ],
    status: 'A'
  },
  {
    _id: 5,
    cust_id: 'Busby Bee',
    ord_date: ISODate("2020-03-19T00:00:00.000Z"),
    price: 50,
    items: [ { sku: 'chocolates', qty: 5, price: 10 } ],
    status: 'A'
  },
  {
    _id: 6,
    cust_id: 'Cam Elot',
    ord_date: ISODate("2020-03-19T00:00:00.000Z"),
    price: 35,
    items: [
      { sku: 'carrots', qty: 10, price: 1 },
      { sku: 'apples', qty: 10, price: 2.5 }
    ],
    status: 'A'
  },
  {
    _id: 7,
    cust_id: 'Cam Elot',
    ord_date: ISODate("2020-03-20T00:00:00.000Z"),
    price: 25,
    items: [ { sku: 'oranges', qty: 10, price: 2.5 } ],
    status: 'A'
  },
  {
    _id: 8,
    cust_id: 'Don Quis',
    ord_date: ISODate("2020-03-20T00:00:00.000Z"),
    price: 75,
    items: [
      { sku: 'chocolates', qty: 5, price: 10 },
      { sku: 'apples', qty: 10, price: 2.5 }
    ],
    status: 'A'
  },
  {
    _id: 9,
    cust_id: 'Don Quis',
    ord_date: ISODate("2020-03-20T00:00:00.000Z"),
    price: 55,
    items: [
      { sku: 'carrots', qty: 5, price: 1 },
      { sku: 'apples', qty: 10, price: 2.5 },
      { sku: 'oranges', qty: 10, price: 2.5 }
    ],
    status: 'A'
  },
  {
    _id: 10,
    cust_id: 'Don Quis',
    ord_date: ISODate("2020-03-23T00:00:00.000Z"),
    price: 25,
    items: [ { sku: 'oranges', qty: 10, price: 2.5 } ],
    status: 'A'
  }
```

Example 2 -Map Reduce

```
var mapFunction1 = function()
 {
   emit(this.cust_id, this.price);
};
```

mapFunction1

```
var reduceFunction1 =
function(keyCustId, valuesPrices)
{
   return Array.sum(valuesPrices);
};
```

reduceFunction1

Example 2 -Map Reduce

```
db.orders.mapReduce(
  mapFunction1,
  reduceFunction1,
  { out: "map_reduce_example" }
);
```

Apply the mapFunction1 and reduceFunction1 functions.

Output the result to another collection (map_reduce_example)

```
db.map_reduce_example.find().sort( { _id: 1 } );
```

map_reduce_example collection

```
test> db.map_reduce_example.find().sort( { _id: 1 } );
[
  { _id: 'Ant O. Knee', value: 95 },
  { _id: 'Busby Bee', value: 125 },
  { _id: 'Cam Elot', value: 60 },
  { _id: 'Don Quis', value: 155 }
]
```

Example 2 -Map Reduce

```
db.orders.aggregate([
  { $group: { _id: "$cust_id", value: { $sum: "$price" } }
},
  { $out: "agg_alternative_1" }
]);
```

```
db.agg_alternative_1.find().sort( { _id: 1 } );
```

```
test> db.agg_alternative_1.find().sort( { _id: 1 } );
[
  { _id: 'Ant O. Knee', value: 95 },
  { _id: 'Busby Bee', value: 125 },
  { _id: 'Cam Elot', value: 60 },
  { _id: 'Don Quis', value: 155 }
]
```

Benefits

- The output of a reduced dataset can be reduced by another reduce function and so on….
- Or, by using aggregate function, we can define queries based on $match, $unwind, $group, etc…
- This helps in analysing millions/billions of records and condensing results.

https://www.mongodb.com/docs/manual/tutorial/map-reduce-examples/

Map Reduce

- Let us try using Map Reduce with the unicorns collection.
- Count all unicorns whose weight > = 600
- Display these unicorns in a separate collection

MapReduce

- Map Function will emit 1 if weight > = 600
- Reduce Function will count all the 1s
- This will be saved in a collection heavy_unicorns
- Display contents of the heavy_unicorns collection

Map

Identify the key and apply your reasoning to this key

```
var mapUnicorn1 = function() {
var key = this.weight;
if(this.weight >= 600)
emit(key, 1);
};
```

Reduce

Reduce the values obtained from the map function

```
var reduceUnicorn1 = function(key, values) {
var sum = 0;
values.forEach(function(value) {
sum += 1;
});
return sum;
};
```

mapReduce/ aggregate

Call the map and reduce functions

```
db.unicorns.mapReduce(
                    mapUnicorn1,
                    reduceUnicorn1,
                    {out: "heavy_unicorns" }
                  );
db.heavy_unicorns.find();
```

```
test> db.heavy_unicorns.find();
[
  { _id: 601, value: 1 },
  { _id: 600, value: 1 },
  { _id: 650, value: 1 },
  { _id: 984, value: 1 },
  { _id: 704, value: 1 },
  { _id: 733, value: 1 },
  { _id: 690, value: 1 }
]
```

Call aggregate

```
db.unicorns.aggregate([
          { $match: { weight: { $gte: 600} } } ,
          { $group: { _id: "$weight", "count": { $sum:1} } },
   { $out: "unicorn_heavy" }
]);
db.unicorn_heavy.find(););
```

```
test> db.unicorn_heavy.find();
[
  { _id: 733, count: 1 },
  { _id: 600, count: 1 },
  { _id: 690, count: 1 },
  { _id: 650, count: 1 },
  { _id: 984, count: 1 },
  { _id: 601, count: 1 },
  { _id: 704, count: 1 }
]
```

Map Reduce/ Aggregate

- When we use “Group By” statements in SQL or LINQ for a collection, we are actually reducing the overall collection information into some summary information.

- This is similar to aggregate or map & reduce functions.

- Information is reduced/aggregated/summarized for it to be interpreted by humans

LINQ Group by operator

Display the total number of staff in each branch

```
var myQuery =
from b in Branches join s in Staff
on b.BrBranchno equals s.StBrBranchno
group s.StBrBranchno by b.BrBranchname into g
select new
{
BranchNo = g.Key,
TotalStaff = g.Count()
} ;
myQuery.Dump();
```

Query<> (8 items)

BranchNo	TotalStaff
Memorial	3
Inner Loop	2
Northwest	2
The Woodlands	2
Round Top	1
Fort Bend	2
Brenham	3
West	2
	17

```
1 select br_branchname, count(st_staffno) "Number of Staff"
2 from staff, branch
3 where st_br_branchno = br_branchno
4 group by br_branchname
```

```
BR_BRANCHNAME        Number of Staff
-------------------- ----------------
Memorial             3
Inner Loop           2
Northwest            2
The Woodlands        2
Round Top            1
Fort Bend            2
Brenham              3
West                 2

8 rows selected
```

LINQ

- Note that grouping is being done by identifying a key
- Next aggregation/grouping is done – count, average, etc

```
var myQuery =
from b in Branches join s in Staff
on b.BrBranchno equals s.StBrBranchno
group s.StBrBranchno by b.BrBranchname into g
select new
{
BranchNo = g.Key,
TotalStaff = g.Count()
} ;
myQuery.Dump();
```

MapReduce

- Note that mapping is being done by identifying a key or matching some value.
- Next aggregation/grouping reduction is done – count, average, etc

1. Map Function
2. Reduce Function

Or use aggregation pipeline

Parallel Processing

- Use of more than one CPU simultaneously to execute a program to make it run faster because each CPU executes different parts of the overall task without interfering with each other.
- Most computers are single-CPU, but some models may have several. With single-CPU computers, we perform parallel processing by connecting the computers in a network. Also called distributed processing.
- Parallel processing differs from multitasking, in which a single CPU executes several programs at once.

Parallel Processing

- **Sharing of load:**
  - Divide the load (dataset) into different portions
  - Execute each portion by a separate CPU. So, each CPU will tackle the task (from the same dataset) at the same time (i.e., parallelly and not sequentially) independently
  - These CPUs are placed in a cluster after considering work-load and load-sharing issues.

Tutorial Exercise

- Use *MapReduce* function/ Aggregate Pipeline to process the data



This is the LAST week for tutorials.